

Análisis de Algoritmos

Ciencias de la Computación

Abril – Agosto / 2025



Unidad 2

2. Notación asintótica

2.1 Notación para el "orden de"

2.2 Notación Omega

2.3 Notación Theta

2.4 Notación asintótica condicional



Semana 4

Notación asintótica

- Saber cómo crece el tiempo de ejecución, cuando el tamaño de la entrada crece. Esto es la eficiencia asintótica del algoritmo.
- “Asintótica” porque analiza el **comportamiento de las funciones en el límite**, es decir, su tasa de crecimiento.
- Se describe por medio de una función cuyo dominio es, los números naturales (N)
- La notación asintótica captura el comportamiento de la función para valores grandes de N .

Notación asintótica

- El tiempo de ejecución de un algoritmo depende de cuánto tiempo le tome a una computadora ejecutar las líneas de código del algoritmo.
- Depende de:
 - La velocidad de la computadora,
 - El lenguaje de programación y el compilador
 - Otros factores.

Notación asintótica

- Tiempo de ejecución en base a dos ideas:
 - ¿Cuánto tiempo se tarda el algoritmo, en términos del tamaño de su entrada?
 - Qué tan rápido crece una función con el tamaño de la entrada. A esto lo llamamos la **tasa de crecimiento** del tiempo de ejecución. Para esto:

*Tenemos que **simplificar** la función para extraer la **parte más importante** y dejar de lado las partes menos importantes.*

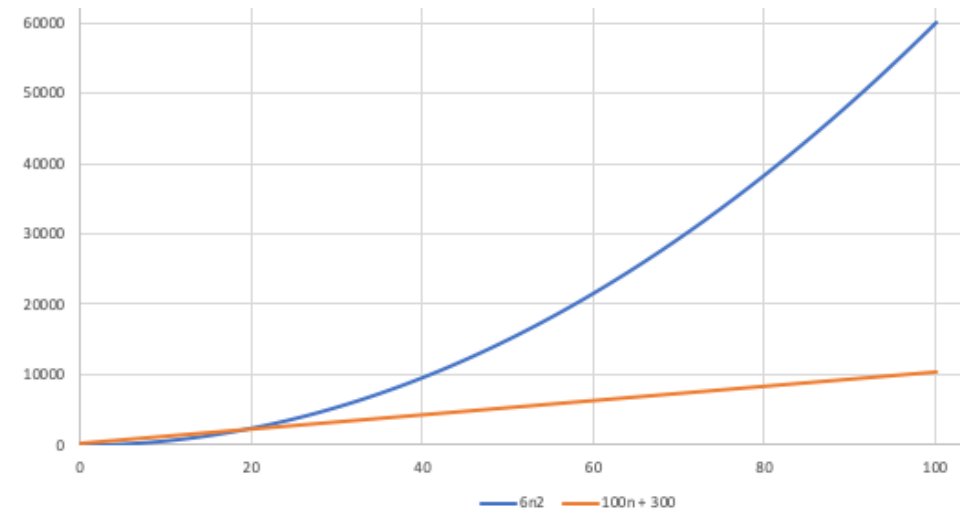
Notación asintótica

- Ejemplo:
 - Supón que un algoritmo, que corre con un entrada de tamaño n , se tarda $6n^2 + 100n + 300$ instrucciones de máquina.
 - Eliminar los **términos no dominantes**. (demostrar)

Notación asintótica

- Ejemplo:

Supón que un algoritmo, que corre con un entrada de tamaño n , se tarda $6n^2 + 100n + 300$ instrucciones de máquina. El término $6n^2$ se vuelve más grande que el resto de los términos, $100n + 300$, una vez que n se hace suficientemente grande, 20 aproximadamente.



Notación asintótica

- Diríamos que el tiempo de ejecución de este algoritmo crece como n^2 , descartando el coeficiente **6** y los términos restantes **$100n + 300$** .
- En realidad no importa qué coeficiente usemos; siempre que el tiempo de ejecución sea **$an^2 + bn + c$** , para algunos números **$a > 0$, b , y c** , siempre habrá un valor de **n** para el cual **an^2** sea mayor que **$bn+c$** , y esta diferencia aumenta a medida que **n** aumenta.

Notación asintótica

- Al descartar los términos menos significativos y los coeficientes constantes, podemos enfocarnos en la **parte importante del tiempo de ejecución** de un algoritmo (*su tasa de crecimiento*).
- Cuando descartamos los coeficientes constantes y los términos menos significativos, usamos **notación asintótica**.

Notación “para el orden de”

- Métrica para determinar la eficiencia de los algoritmos.
- Por ejemplo, decir que el tiempo de ejecución $T(n)$ de un programas en $O(n^2)$.
 - Se lee: “**o mayúscula de n al cuadrado**” o tan solo “**o de n al cuadrado**”.
 - Significa: que existen constantes enteras positivas c y n_0 tales que para n mayor o igual que n_0 , se tiene que:

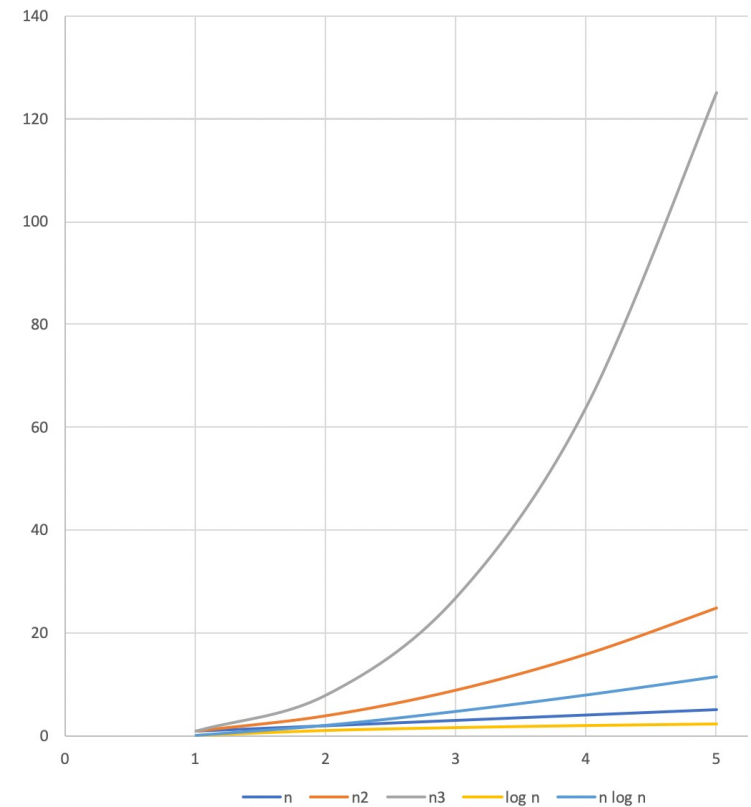
$$T(n) \leq cn^2$$

Notación “para el orden de”

- Se dice que $O(f(n))$ define un "orden de complejidad".
- Así tendremos:
 - Orden constante: $O(1)$
 - Orden Logaritmico: $O(\log n)$
 - Orden Lineal: $O(n)$
 - Orden cuadrático: $O(n^2)$
 - Orden exponencial. $O(a^n)$ ($a > 2$)
 - Orden factorial: $O(n!)$

Notación “para el orden de”

- La mejor técnica para diferenciar la eficiencia de los algoritmos es el estudio de los **órdenes de complejidad**.
- En el análisis de algoritmos se considera usualmente el **caso peor**.



Notación “para el orden de”

- Aspectos importantes:
 - Si un programa se va a utilizar sólo algunas veces, el costo de su escritura y depuración es el dominante, rara vez el tiempo de ejecución influye en el costo total.
 - Si se van a utilizar entradas pequeñas, la velocidad de crecimiento de tiempo de ejecución puede resultar irrelevante.
 - En algoritmos numéricos, la precisión y la estimación son tan importantes como la eficiencia.

Notación “para el orden de”

P1

$$T_1(n) \longrightarrow O(f(n))$$

P2

$$T_2(n) \longrightarrow O(g(n))$$

Regla del máximo

$$T_1(n) + T_2(n) = O(\max(f(n), g(n)))$$

Notación “para el orden de”

- **Ejemplo:**

- Supón que tiene 3 pasos específicos de un programa cuyos tiempos de ejecución son:

$$O(n^2), O(n^3) \quad \text{y} \quad O(n \log n)$$

- Entonces, el tiempo de ejecución de los dos primeros pasos ejecutados en secuencia es:

$$= O(\max(n^2, n^3))$$

$$= O(n^3)$$

Notación “para el orden de”

- El tiempo de ejecución de los tres juntos es:

$$= O(\text{máx}(n^3, n \log n))$$

$$= O(n^3)$$

En conclusión:

El tiempo de ejecución de una secuencia fija de pasos, dentro de un factor constante, es igual al **tiempo de ejecución del paso con mayor tiempo de ejecución**.

Notación “para el orden de”

- En raras ocasiones dos pasos pueden tener tiempos de ejecución inusuales.
 - Por ejemplo:

$$f(n) = \begin{cases} n^4 & \text{si } n \text{ es par} \\ n^2 & \text{si } n \text{ es impar} \end{cases} \quad g(n) = \begin{cases} n^2 & \text{si } n \text{ es par} \\ n^3 & \text{si } n \text{ es impar} \end{cases}$$

$$O(\max(f(n), g(n))) \quad n^4 \text{ si } n \text{ es par y } n^3 \text{ si } n \text{ es impar}$$

Notación “para el orden de”

- Matemáticamente se representa:

$$O(f(n)) = \{t: N \rightarrow R^{\geq 0} \mid (\exists c \in R^+)(\forall n \in N)[t(n) \leq cf(n)]\}$$

El conjunto de todas las funciones $t: N \rightarrow R^{\geq 0}$

tales que $t(n) \leq cf(n)$

para todo $n \geq n_0$

para una constante positiva real c y para un umbral entero n_0

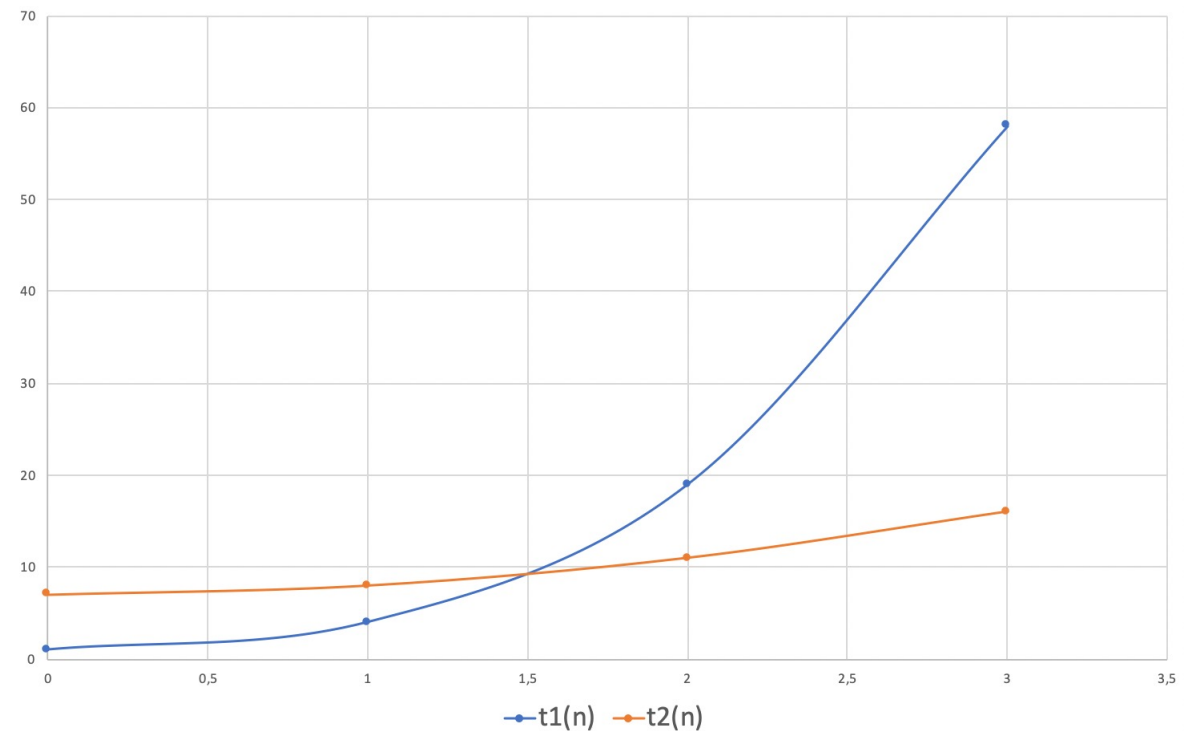
Notación “para el orden de”

- Supongamos dos funciones $T_1(n)$ y $T_2(n)$ que son representativos de dos algoritmos que solucionan un mismo problema.
 - Si $T_1(n) = 2n^3 + n + 1$ $T_2(n) = 42n^2 + 7$
 - Entonces $T_1(n) = O(n^3)$ $T_2(n) = O(n^2)$

Concluye que $T_2(n)$ tiene mejor comportamiento computacional que $T_1(n)$ para valores de n suficientemente grandes.

Notación “para el orden de”

n	t1(n)	t2(n)	relación
0	1	7	7,00
1	4	8	2,00
2	19	11	0,58
3	58	16	0,28



Notación “para el orden de”

- Práctica: Demostrar para valores n grandes

Ejercicio

```
n = 2
contador = 0
numero = 2

while (contador < n)
    sumaDivisores = 1;
    for (int i=2 to i<=numero/2)
        if (numero % i == 0)
            sumaDivisores = sumaDivisores+i
    if (sumaDivisores == numero)
        Presentar (numero)
        contador = contador + 1
    numero = numero + 1
```



Semana 5

Notación “para el orden de”

$$t(n) = 27n^2 + \frac{355}{113}n + 12$$

$$O(n^2) \quad O(n) \quad O(1)$$

$$n^2 \geq n \quad n \geq 1$$

$$n^2 \geq 1$$

$$= 27 + \frac{355}{113} + 12$$

$$c = \frac{4762}{113}$$

$$t(n) \leq cf(n)$$

$$27n^2 + \frac{355}{113}n + 12 \leq \frac{4762}{113}(n^2)$$

$$n \geq 1$$

$t(n)$ está en el orden de $(f(n))$

¿ $f(n)$ está en el orden de $(t(n))$?

Notación

- Regla del umbral
 - $t(n) \in O(f(n))$ si y sólo si existe una constante real positiva, para cada numero natural n .
- Regla de máximos:
 - $O(f(n) + g(n)) = O(\text{máx}(f(n), g(n)))$

Notación Omega

- La notación $\Omega(f(n))$ se utiliza para describir una **cota inferior asintótica**.
- Significa que el algoritmo **no puede ejecutarse más rápido que una cierta función $f(n)$** en el mejor de los casos, cuando el tamaño de la entrada es lo suficientemente grande

Notación Omega

- **Definición formal:**

- Un algoritmo es $\Omega(f(n))$ si existen constantes positivas d y n_0 tales que, para todo $n \geq n_0$, se cumple:

$$T(n) \geq cf(n)$$

- **Dónde:**

- $T(n)$ es el tiempo de ejecución del algoritmo para una entrada de tamaño n .
- d es una constante positiva,
- n_0 es el tamaño mínimo de entrada a partir del cual se cumple la desigualdad.

Notación Omega

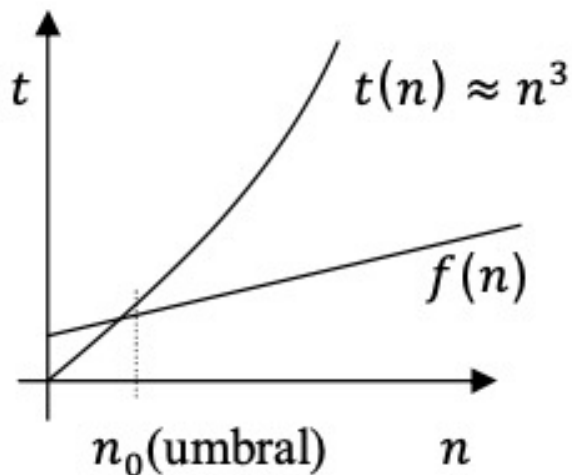
- Permite establecer el mejor tiempo de ejecución posible de un algoritmo.
- Es útil para establecer límites mínimos de eficiencia que ningún algoritmo puede superar para cierto problema.
- También se usa para mostrar que ciertos problemas no pueden resolverse más rápido que un cierto orden de complejidad.

Notación Omega

Matemáticamente, esto significa que existe una constante real positiva d y un umbral entero n_0 tal que $t(n) \geq d * f(n)$ siempre que $n \geq n_0$.

$$\Omega(f(n)) \equiv \{t: \mathbb{N} \rightarrow \mathbb{R}^{\geq 0} \mid \exists d \in \mathbb{R}^+, n_0 \in \mathbb{N}, \forall n \geq n_0 \mid t(n) \geq d * f(n)\}$$

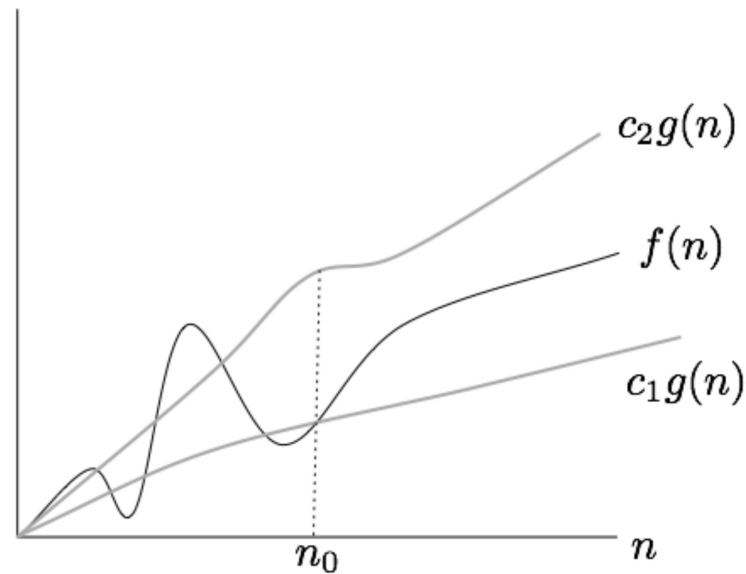
Gráficamente sería:



$t(n) \in \Omega(f(n))$: Cota inferior.

$f(n) \in O(t(n))$: Cota superior.

Notación Theta



$$f(n) = \Theta(g(n))$$

$$\{t: \mathbb{N} \rightarrow \mathbb{R}^{>0} \mid (\exists c, d \in \mathbb{R}^+)(\forall n \in \mathbb{N})[df(n) \leq t(n) \leq cf(n)]\}$$

Notación Omega

```
Función SumaNumeros(n)
  s <- 0
  Para i de 1 a n
    s <- s + i
  Devuelve s
Fin Función
```

- El bucle se ejecuta exactamente n veces, independientemente de cualquier otro factor.
- En cada iteración del bucle, se realiza al menos una operación (la adición y la asignación).

Dado este comportamiento, podemos afirmar que el número de operaciones realizadas por el algoritmo es al menos n . Por lo tanto, la función tiene una complejidad de tiempo que es $\Omega(n)$. Esto se puede formalizar diciendo que existe una constante c (por ejemplo, $c = 1$) y un n_0 (por ejemplo, $n_0 = 1$), tal que para todo $n \geq n_0$, el número de operaciones es al menos $c \cdot n$.

Notación Theta

- Se utiliza el símbolo Θ
- Acotar el tiempo de ejecución de un algoritmo tanto por encima como por debajo mediante multiplos reales positivos posiblemente distintos de una misma función.
- $t(n)$ está en Theta de $f(n)$ o $t(n)$ está en el orden exacto de $f(n)$
 - Se denota: $t(n) \in \Theta(f(n))$
 - $\Theta(f(n)) = O(f(n)) \cap \Omega(f(n))$

Notación

- Notación O

- $O(f(n)) = \{t: \mathbb{N} \rightarrow \mathbb{R}^{>0} \mid (\exists c \in \mathbb{R}^+)(\forall n \in \mathbb{N})[t(n) \leq cf(n)]\}$

- Notación Omega

- $\Omega(f(n)) = \{t: \mathbb{N} \rightarrow \mathbb{R}^{>0} \mid (\exists d \in \mathbb{R}^+)(\forall n \in \mathbb{N})[t(n) \geq df(n)]\}$

- Notación Theta

- $\Theta(f(n)) = O(f(n)) \cap \Omega(f(n))$

- $\{t: \mathbb{N} \rightarrow \mathbb{R}^{>0} \mid (\exists c, d \in \mathbb{R}^+)(\forall n \in \mathbb{N})[df(n) \leq t(n) \leq cf(n)]\}$

Ejemplos

- Dado el siguiente:

$$t(n) = 2n + 10$$

Determine si:

$$t(n) \in O(n)?$$

Demostración

- Regla del límite
 - Nos permite comparar dos funciones en cuanto a la notación asintótica se refiere.
 - Se debe calcular el siguiente límite:
 - $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$

Demostración

- Se dan 3 resultados:

$$1. \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c \in R \Rightarrow$$

$$\Rightarrow \left\{ \begin{array}{lll} f(n) \in O(g(n)) & f(n) \in \Omega(g(n)) & f(n) \in \theta(g(n)) \\ g(n) \in O(f(n)) & g(n) \in \Omega(f(n)) & g(n) \in \theta(f(n)) \end{array} \right\}.$$

Estas funciones se comportan igual. Se diferencian en una constante multiplicativa.

Demostración

$$2. \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty \Rightarrow$$

$$\Rightarrow \left\{ \begin{array}{lll} f(n) \notin O(g(n)) & f(n) \in \Omega(g(n)) & f(n) \notin \theta(g(n)) \\ g(n) \in O(f(n)) & g(n) \notin \Omega(f(n)) & g(n) \notin \theta(f(n)) \end{array} \right\}.$$

Por muy alta que sea la constante multiplicativa de $g(n)$ nunca superará a $f(n)$.

Demostración

$$3. \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0 \Rightarrow$$

$$\Rightarrow \left\{ \begin{array}{lll} f(n) \in O(g(n)) & f(n) \notin \Omega(g(n)) & f(n) \notin \theta(g(n)) \\ g(n) \notin O(f(n)) & g(n) \in \Omega(f(n)) & g(n) \notin \theta(f(n)) \end{array} \right\}.$$

$g(n)$ crece más exponencialmente que $f(n)$. Sería su cota superior.

Ejemplo

1. ¿Qué significa que el tiempo de ejecución de un algoritmo está “en el orden excto de $f(n)$?”
2. Demostrar que:
 - $T(n) = 5 * 2^n + n^2$ está en el orden exacto de 2^n

Ejemplo

1. Significa:

Diremos que $t(n)$ está en Theta de $f(n)$, o lo que es igual que $t(n)$ está en el orden exacto de $f(n)$ y lo denotamos $t(n) \in \theta(f(n))$, si $t(n)$ pertenece tanto a $O(f(n))$ como a $\Omega(f(n))$.

Ejemplo

2. Utilizamos la regla del límite

$f(n) = 5 * 2^n + n^2$ y $g(n) = 2^n$. Pasamos a resolver el límite, como sigue:

$$\lim_{n \rightarrow \infty} \frac{5*2^n + n^2}{2^n}$$

Al resolverla llegamos a una indeterminación, por lo que aplicaremos el **teorema de L'Hôpital** tantas veces como sea necesario hasta llegar a una conclusión coincidiendo con cualquiera de los casos anteriores:

$$\begin{aligned} \lim_{n \rightarrow \infty} \frac{5*2^n + n^2}{2^n} &= \lim_{n \rightarrow \infty} \frac{5*2^n * \log(2) + 2*n}{2^n * \log(2)} = \lim_{n \rightarrow \infty} \frac{5*2^n * \log(2)^2 + 2}{2^n * \log(2)^2} = \\ \lim_{n \rightarrow \infty} \frac{5*2^n * \log(2)^3}{2^n * \log(2)^3} &= 5. \end{aligned}$$

Observamos que al final sería una constante numérica, por lo que estamos en el caso 1, concluyendo que $f(n) \in \theta(g(n))$ y, por tanto, queda demostrado.

Taller

- Dado: $f(n) = n^3 + 9n^2 \log(n)$ y $g(n) = n^2 \log(n)$
 - Comprobar $f(n) \in O(g(n))$
 - Comprobar $f(n) \notin O(n^2)$
- Demostrar formalmente si existe relación de pertenencia entre $f(n)$ y $O(g(n))$ y también entre $g(n)$ y $O(f(n))$ considerando $f(n) = 2^n$ y $g(n) = 2^{2n}$